

From Scratch:
The Design and Implementation of a SAT Solver
From Scratch

Oliver Lie

August 1, 2026

Contents

Chapter 1	Introduction	4
1.1	What Do SAT Solvers Even Solve?	5
1.2	CNF Encoding	6
1.3	Reading a CNF Equation Programmatically	9
Chapter 2	Brute Force	13
2.1	Optimizing Brute Force	15
2.2	Brute Force – Results	18
Chapter 3	DPLL	19
3.1	Recursive Backtracking	21
3.1.1	Recursive Backtracking – Results	25
3.2	Unit Propagation	25
3.2.1	Changes To Our Implementation	26
3.2.2	“Textbook” Unit Propagation	26
3.3	Better Unit Propagation	30
3.3.1	Better Unit Propagation - Results	35
3.4	Pure Literal Elimination	36
3.5	Optimizing Our Current Approach	36
3.6	Iterative DPLL	36

Preface

Hello! Thank you for taking me seriously enough to pick up this book and begin reading it. Before we continue, I just want you to know that I am not an expert when it comes to things such as algorithms, data structures, general computer science, or even SAT solvers (what this book is about). Despite that, that’s exactly why I wanted to write this. These things that I am not an expert in are my passion, and by doing this technical research, I hope to become a little less of “not an expert” in those fields.

The aim of this book isn’t to improve or revolutionize SAT solving algorithms. Rather, it is to make the methods and reasoning more accessible to the lay undergraduate (I am open to the possibility that I myself may be the lay undergraduate, and you are simply smarter than me).

Throughout this book we will build our own modern SAT solver from scratch, complete with all the bells and whistles. We will begin with the most basic algorithm imaginable, brute force, and gradually work our way through DPLL, CDCL, and beyond.

Throughout this journey we will encounter thought experiments, exercises, and a handful of experiments intended to solidify our understanding. Any external resources will be cited (hopefully correctly), and in keeping with the goal of accessibility:

- Questions that are explicitly asked will eventually be answered.
- Proofs will be explained in informal language before becoming formal.
- Code will be provided and linked through GitHub (or another hosting platform).
- Revisions to this book will be made whenever I discover mistakes or better explanations.

There is one more thing that deserves acknowledgement: some pieces of code were AI-assisted. To be completely transparent, “AI-assisted” means that instead of spending days repeatedly banging my head against a wall trying to understand a bug or some subtle algorithmic detail, I occasionally asked an AI to explain concepts or help debug an implementation. Every word of this book, however, is my own. AI did not generate any of the explanatory text

you will read here, although it certainly helped produce some of the code that accompanies it.

There are practical reasons for this. SAT solvers are difficult to implement correctly, and introducing subtle bugs is remarkably easy. In fact, before deciding to write this book, I had already attempted to build a SAT solver once before. I never managed to get a working CDCL implementation.

That failure, however, taught me a tremendous amount. So in a sense, we are learning SAT solving together (isn't that comforting?). My previous mistakes have shown me many of the pitfalls, and my hope is that by documenting this journey I can help you avoid making the same ones. So without further ado, let's begin. By the time we reach the end, I hope I will have taught you more than I myself learned.

Chapter 1

Introduction

SAT solvers, in my opinion, are one of the most underrated pieces of algorithmic technology of the modern day. They are used in both software and hardware verification (circuitry testing), product configuration, package management, computational biology, particle physics, solving graph problems, and much much more.¹

In a more computer science-focused world, they have applications in NP-Completeness. Since (nearly) every problem has a reduction to a Boolean SAT problem, creating one really really good SAT solver lets us solve all those problems in a “fast” manner. So they are important everywhere despite the vast majority of us never really thinking about their existence. One note on what “fast” means: even if solving a SAT problem takes a small amount of time, the algorithmic complexity isn’t necessarily “fast” as there is no known polynomial algorithm for any NP-Complete problem (I am a firm believer that $P = NP$, and yes I know that makes me crazy).

Before we start writing out code, proofs, and the like, we need to make sure we’re all on the same page on things such as input and ideas. Reading through this introduction is important so that you understand the basic notation used throughout this book.

One last thing to note: we will write out pseudocode rather than the code for one specific programming language. However, although the idea behind pseudocode is that it’s programming language agnostic (it doesn’t carry artifacts of any real programming language (usually)), this idea is poorly executed when it comes to formatting. My algorithms professor will probably scream at me for this, but I’m making the executive decision that we will follow a general C-based syntax for readability purposes. That is, we will use braces `{}` for block scope, parentheses `()` for grouping, `=` for assignment, `==` for equality, and `<=`, `>=`, etc. for their respective uses. Everything that can be inferred visually will be used. Another thing to note is that we will pretend our arrays can dynamically resize, so we won’t pre-allocate space, and we will also assume 1-based indexing for an

¹<https://www.cs.utexas.edu/~isil/cs389L/lecture4-6up.pdf>

easier map from variables to indices, as you will see later.

1.1 What Do SAT Solvers Even Solve?

“What do SAT solvers even solve,” I hear you say? SAT solvers solve *boolean satisfiability* problems. Another way to say that is they solve *propositional logic* problems. There are many forms of propositional logic, and you may be familiar with it. Here are some basic examples:

- $P \rightarrow Q$, an implication statement
- $(A \wedge B)$, an and statement
- $(A \vee B)$, an or statement
- $\neg A$, a negation/not statement

Of course these are very basic examples, and we haven’t included every operator such as xor, nand, etc. As you know, we can rewrite implications to contain only and, or, and not operators, which is what SAT solvers operate on. In other words, SAT solvers only operate on Boolean algebra equations.²

However, SAT solvers don’t operate on just any Boolean algebra equation. No no no, they must be in a proper form, called CNF (more on that in the next section). CNF (Conjunctive Normal Form) equations are a “conjunction of disjunctions,” i.e., clauses of ors, combined with ands.

Some examples of CNF are as follows:

- $(A \vee B)$
- $(A \vee B) \wedge (\neg C)$
- $(A \vee B) \wedge (\neg C \vee D)$

Some examples that are **not** CNF:

- $\neg(A \vee B)$, where there is a NOT in front of a clause
- $\neg(A \vee B) \wedge (C)$, where there is a NOT in front of a clause
- $(A \vee B \wedge C)$, where there is an AND within a clause
- $(A \wedge B) \vee (C)$, where it is in DNF form.³

²A Boolean algebra equation is one containing only and ($\wedge$), or ($\vee$), and not ($\neg$) operations (at a basic level).

³DNF (Disjunctive Normal Form) is a “disjunction of conjunctions,” i.e., clauses of ands combined by ors. Solving these is trivially easy, as we just need to find one clause where no element evaluates to false.

1.2 CNF Encoding

Now that we know what our inputs look like, the next question (I hope you're asking questions) is "how do we encode that?" Well good thing you're reading this, because that's what we're here to talk about! In this section, we'll expand our base of boolean algebra, including some definitions, transformations, and finally good encoding.

Definitions We'll Use

Clause A clause is a collection of variables, in which each variable is separated by an or ($\vee$), and each clause is separated by an and ($\wedge$).

Variable A variable is an element of a clause, taking on a value of True, False, or Unassigned. Each variable can be negated ($\neg$) or not, affecting its evaluated value.

Literal A literal is an evaluated variable, evaluating to one of True, False, or Unassigned. Since a literal is an evaluated variable, it cannot be negated. A literal whose corresponding variable is unassigned evaluates to Unassigned, regardless of whether or not the variable is negated.

A quick example of the difference of a literal value, using all three of our definitions:

$$(\neg A)$$

is a clause of one variable "A", which is negated. If the variable $A = \text{False}$, then its literal value is *True*, because

$$\neg \text{False} \equiv \text{True}.$$

In other words, A was assigned the value False, and then it evaluated to True. This will be very important in the future, so please take the time to understand it well enough.

As discussed earlier, a CNF equation only has three operators: and ($\wedge$), or ($\vee$), and not ($\neg$), but in propositional logic, there are many more operators, and their clauses are not guaranteed to be in the format we want. There are many algorithms to transform any propositional logic equation into CNF, but frankly, that's beyond the scope of this book (it's also beyond my scope). Instead, we will just look at how we transform each operator into an equivalent format using just our three operators.

Implication ($\rightarrow$)

p	q	$\neg p$	$p \rightarrow q$	$\neg p \vee q$
T	T	F	T	T
T	F	F	F	F
F	T	T	T	T
F	F	T	T	T

Xor ($\oplus$)

p	q	$\neg p$	$\neg q$	$p \oplus q$	$p \vee q$	$\neg p \vee \neg q$	$(p \vee q) \wedge (\neg p \vee \neg q)$
T	T	F	F	F	T	F	F
T	F	F	T	T	T	T	T
F	T	T	F	T	T	T	T
F	F	T	T	F	F	T	F

Biconditional ($\leftrightarrow$)

p	q	$p \leftrightarrow q$	$p \rightarrow q$	$q \rightarrow p$	$(\neg p \vee q) \wedge (\neg q \vee p)$
T	T	T	T	T	T
T	F	F	F	T	F
F	T	F	T	F	F
F	F	T	T	T	T

We won't look at negations of operators such as nand, nor, xnor, or any others. I'm not a boolean algebra expert when it comes to what operators exist. But if you were given an equation with these extra operators, you could easily apply the correct equivalency, then apply an algorithm that converts any boolean algebra equation into one of CNF form.⁴

DIMACS CNF Format

Now that we know what CNF looks like, we can now discuss how it's formatted for a SAT solver's consumption. Typically, CNF is formatted in *DIMACS CNF* form.

Some rules for DIMACS CNF form are:

1. The file may begin with comment lines. The first character of each comment line must be a lower case letter "c". Comment lines typically occur in one section at the beginning of the file, but are allowed to appear throughout the file.
2. The comment lines are followed by the "problem" line. This begins with a lower case "p" followed by a space, followed by the problem type, which for CNF files is "cnf", followed by the number of variables followed by the number of clauses.
3. The remainder of the file contains lines defining the clauses, one by one.
4. A clause is defined by listing the index of each positive literal, and the negative index of each negative literal. Indices are 1-based, and for obvious reasons the index 0 is not allowed.
5. The definition of a clause may extend beyond a single line of text.

⁴I honestly may be talking out of my ass here. I will do more research on this subject at a later date.

6. The definition of a clause is terminated by a final value of “0”.
7. The file terminates after the last clause is defined.

There are some things about this syntax to be wary of:

1. The definition of the next clause normally begins on a new line, but may follow, on the same line, the “0” that marks the end of the previous clause.
2. In some examples of CNF files, the definition of the last clause is not terminated by a final “0”.
3. In some examples of CNF files, the rule that the variables are numbered from 1 to N is not followed. The file might declare that there are 10 variables, for instance, but allow them to be numbered 2 through 11.
4. Comments can appear anywhere in the file, but the comment must be on its own line and it must start with “c”.⁵

For a quick example, here is a simple way to format a DIMACS CNF equation:

```
p cnf 4 2
1 2 -3 0
-1 4 0
```

And one with comments:

```
c this is a comment
p cnf 4 2
c this is another comment
1 2 -3 0
-1 4 0
```

In order to increase the compatibility of our SAT solver, we will be bending some rules.

We will firstly allow missing header lines. In the case where no header is passed in, we will instead infer the number of clauses and variables. We will also be very lenient in where comments can appear, but we will enforce that variables must be between 1 and N .

1. Comments can appear anywhere in the file, but they **MUST** be on their own line and start with “c”.
2. A header is not necessary, and if one is not provided, then the number of variables and clauses shall be inferred. However, if one is provided, then the number of variables and clauses **MUST** match the header.
3. All clauses, with the exception of the last clause, must end with “0”, and can extend multiple lines.

⁵<https://jix.github.io/varisat/manual/0.2.0/formats/dimacs.html>

4. All variables are numbered 1 through N , where N is the number of variables. That is, if you say there are 10 variables, they must be labeled $1, 2, \dots, 10$ and not $2, 3, \dots, 11$ or any other convention.
5. (For compatibility) Upon encountering a “%” character outside of a comment, we will stop parsing immediately.

Thus, valid input can take on many forms:

```

p cnf 4 2
1 2 -3 0
-1 4 0

4 2
1 2 -3 0
-1 4 0

p cnf 4 2
c this comment doesn't have to be at the top
1 2 -3 0
-1 4 0

p cnf 4 2
c this comment doesn't have to be at the top
1 2 -3 0
-1 4
c the first two numbers are still the number
c of variables and the number of clauses

p cnf 4 2 1 2 -3 0
-1 4 0
```

More information can be found here.⁶

1.3 Reading a CNF Equation Programmatically

Now that we know what our input looks like, we can finally discuss reading it programmatically. Firstly we must discuss how our SAT solver will be represented in code. At the moment, we just need to keep track of

- the number of variables
- the number of clauses
- each clause in our equation
- whether or not we’ve tried to solve the equation yet
- what our solution is

⁶<https://people.sc.fsu.edu/~jburkardt/data/cnf/cnf.html>

We also want some methods to parse the equation, construct a SAT solver, solve the equation, print out the solution if it is satisfiable, and whether or not there's a solution. Therefore, our object will look like so:

```

1  public interface:
2      SATSolver(string input)
3      bool solveCNF()
4
5  private interface:
6      void parse(string equation)
7      int numVars
8      int numClauses
9      int[] [] clauses
10     optional<bool>[] vars

```

Our pseudocode is as follows:

```

1  SATSolver(string input)
2  {
3      vars = no-value
4
5      if (input is file)
6      {
7          string content = readFile(input)
8          parse(content)
9      }
10     else
11     {
12         parse(input)
13     }
14 }

```

```

1  parse(equation)
2  {
3      // The method of parsing if there's a header
4      if (contains(equation, "p cnf"))
5      {
6          int[1..N] Arr
7
8          for each (line in input)
9          {
10             if (startsWith(line, 'c'))
11             {
12                 skip to next line
13             }
14             else
15             {
16                 for each (token in line)

```

```
17         {
18             append(Arr, token)
19         }
20     }
21 }
22
23 numVars = Arr[1]
24 numClauses = Arr[2]
25
26 idx = 2
27
28 for (i = 1 up to numClauses)
29 {
30     int[1..N] clause
31
32     while (Arr[idx] != 0)
33     {
34         append(clause, Arr[idx])
35         idx = idx + 1
36     }
37
38     append(clauses, clause)
39 }
40
41 // Method of parsing when there's no header
42 else
43 {
44     numVars = 0
45     numClauses = 0
46
47     int[1..N] Arr
48
49     for each (line in input)
50     {
51         if (startsWith(line, 'c'))
52         {
53             skip to next line
54         }
55         else
56         {
57             for each (token in line)
58             {
59                 numVars = max(numVars, token)
60
61                 if (token == 0)
62                 {
63                     numClauses = numClauses + 1
64                 }
65             }
66         }
```

```

67         append(Arr, token)
68     }
69
70     idx = 0
71
72     for (i = 1 up to numClauses)
73     {
74         int[1..N] clause
75
76         while (Arr[idx] != 0)
77         {
78             append(clause, token)
79             idx = idx + 1
80         }
81
82         append(clauses, clause)
83     }
84 }
85 }
86 }
87 }

```

This code will be implemented slightly differently in the example code, as I am using C++. There is also a big bug, which is that the code assumes that because “p cnf” appears in the file, there is a header. This is not always true because it does not consider whether or not the header is on the same line as a comment, so this will break it:

```

c p cnf
1 2 -3 0
-1 4 0

```

Because it assumes that there is a header when in fact there is not. I will leave fixing this bug as an exercise to both the reader and the writer, however, I will not be implementing this fix because if you’re purposely trying to break the constructor, you’re not trying to solve a CNF equation anyways. The fix, in case you’re wondering, would be to just scan the lines and for each line that doesn’t start with “c,” check that it starts with “p cnf,” and if it does, then you know for certain that there is a header. Now the real exercise is to determine whether or not there’s a header in just one pass instead of two.

Chapter 2

Brute Force

Since the dawn of man, whenever encountered with a difficult problem, we have decided “eh, let’s do it in the hardest way possible because I can’t think of a better solution.” The history of brute forcing things goes all the way back to our ancestors discovering fire,¹ all the way to our current solution to climate change.²

The idea of brute forcing something is really as simple as it gets. We have n number of literals, and therefore 2^n possible assignments, and we will check them all until we have either found a solution, or until we’ve run out of assignments to check. Because of how simple the idea is, it would just be better to show code and explain that instead of explaining then showing code.

```
1  solveCNF()
2  {
3      bool[1..numVars] Assignment
4
5      for (i = 0 up to 1 << numVars)
6      {
7          for (j = 0 up to numVars - 1)
8          {
9              Assignment[j] = i & (1 << j)
10         }
11
12         bool solved = true
13
14         for each (clause in clauses)
15         {
16             bool satisfied_clause = false
17
18             for (j = 1 up to clause.size)
19             {
```

¹This is probably not true.

²It’s doing nothing and hoping everything sorts itself out, which is really disappointing.

```

20         int variable = clause[j]
21         int v_idx = abs(variable)
22
23         if (variable > 0)
24         {
25             satisfied_clause = Assignment[v_idx]
26             || satisfied_clause
27         }
28         else
29         {
30             satisfied_clause = !Assignment[v_idx]
31             || satisfied_clause
32         }
33     }
34
35     solved &= satisfied_clause
36 }
37
38 if (solved == true)
39 {
40     vars = Assignment
41     return true
42 }
43 }
44
45 return false
46 }
```

Unfortunately, the formatting across the page kind of sucks, but let's try to explain it anyways.

First, we create an array to hold each boolean value, and then next we start our loop from 0 all the way to “1 $\ll$ numVars.” If you are unfamiliar with what the left bit shift operator (“ $\ll$ ”) is, essentially, it is a fast way of computing some number times 2 times the number of places to shift. That is all the explanation I am going to give because that's not the point. But by computing “1 $\ll$ numVars”, we are computing $1 \rightarrow 2^n$, which happens to be the total number of possible variable combinations (from all False to all True).

Then in the first inner for loop, we put the current variable combination into the ‘Assignment’ array. The index variable ‘i’ holds the current variable combination, and we isolate the j th variable using “1 $\ll$ j” and put it into the j th spot in the ‘Assignment’ array.

Next, we evaluate the CNF equation against the current assignment. We first assume that the equation is solved (innocent until proven guilty), and then we evaluate each clause. We initially assume the clause is false because logically, if one literal is True, then OR-ing it with False makes the “clause satisfied” condition true (we could exit out of the evaluation loop, but it's easier to read if we don't). Then, we “and” the “clause satisfied” evaluation with the “equation solved” condition.

At the end of the evaluation loop, we check if we found a satisfiable assignment. If we have, we then make our ‘vars’ field, which holds the solution if there is one, the assignment, and then we return True to indicate that we have found a satisfiable assignment. If each possible assignment is not satisfiable, then we return False, and ‘vars’ will hold nothing.

Since it’s trivially easy to see why the code works, we will move onto something more interesting: optimizing brute force.

2.1 Optimizing Brute Force

I’ll be completely honest with you, if you want to skip this section, go ahead. It’s more or less just something interesting enough we can do to make the (arguably) worst algorithm³ better.

Since we’re dealing with pseudocode, we’re not going to consider the hardware side of computers such as branch prediction, cache locality, or anything that the hardware uses to execute our code (other than the CPU). We will consider it from a pure Big-O perspective (excluding constant factors unless we’re choosing between two algorithms with the same computational complexity).

The first thing we will add is short circuiting to our clause evaluation. After line 26 where we finish OR-ing the “clause satisfied” boolean, we will add in a new block:

```

1  ...
2  bool satisfied_clause = false;
3
4  for (j = 1 up to clause.size)
5  {
6      int variable = clause[j]
7      int v_idx = abs(variable)
8
9      if (variable > 0)
10     {
11         satisfied_clause =
12             Assignment[v_idx] || satisfied_clause
13     }
14     else
15     {
16         satisfied_clause =
17             !Assignment[v_idx] || satisfied_clause
18     }
19
20     if (satisfied_clause == True)
21     {
22         continue
23     }
24 }
```

³I do not know of any algorithm worse than brute forcing.


```

25
26  ...

```

This just says “once this clause is satisfied, go evaluate the next one immediately.”

We can then add another conditional after this loop, which will move onto the next variable assignment the instant the equation is no longer satisfied:

```

1  ...
2  bool solved = true;
3
4  for each (clause in clauses)
5  {
6      bool satisfied_clause = false;
7
8      for (j = 1 up to clause.size)
9      {
10         ...
11     }
12
13     if (satisfied_clause == True)
14     {
15         continue
16     }
17
18     solved &= satisfied_clause;
19
20     if (solved == False)
21     {
22         break
23     }
24 }
25
26 ...

```

The next algorithm optimization we can make is removing the loop where we move the current assignment into the ‘Assignment’ array. Rather than doing it upon every iteration, why don’t we do it once we find a satisfiable assignment?

```

1  solveCNF()
2  {
3      bool[1..numVars] Assignment
4
5      for (i = 0 up to 1 << numVars)
6      {
7          bool solved = true
8
9          for each (clause in clauses)
10         {

```

```
11         bool satisfied_clause = false
12
13         for (j = 1 up to clause.size)
14         {
15             int variable = clause[j]
16             int v_idx = abs(variable)
17
18             if (variable > 0)
19             {
20                 satisfied_clause = (i & (1 << j))
21                 || satisfied_clause
22             }
23             else
24             {
25                 satisfied_clause = !(i & (1 << j))
26                 || satisfied_clause
27             }
28
29             if (satisfied_clause == True)
30             {
31                 continue
32             }
33         }
34
35         solved &= satisfied_clause
36
37         if (solved == False)
38         {
39             break
40         }
41     }
42
43     if (solved == True)
44     {
45         for (j = 0 up to numVars - 1)
46         {
47             Assignment[j] = i & (1 << j)
48         }
49
50         vars = Assignment
51         return true
52     }
53 }
54
55 return false
56 }
```

With that, that's all the optimization I can think of. In case you're wondering, no this does not change the asymptotic runtime of the method, however, it's

easy to see why these changes would presumably make the algorithm run faster as we’re adding code that stops evaluation the moment we’ve figured just enough out to know if it’s worth continuing evaluation and deferring extra calculation (recording the current assignment) until once we know it’s satisfiable.

As an exercise to the reader, I will ask you, how can you parallelize this algorithm? And once you’ve figured that out, can you program it in a way where once one thread finds a satisfactory assignment, it terminates all other threads immediately instead of waiting for the others to finish?

2.2 Brute Force – Results

To see how well our current algorithm does, running some tests would be a good idea. Why run tests when we have already proven that our algorithm works “perfectly?” To see how quickly it runs of course.

To test its speed (and as we add more features, to ensure correctness is kept), we use a database of problems from here:

<https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html>

We gave our current implementation 1000 SAT instances (uf20-91), each with 20 variables and 91 clauses each, all satisfiable. It took this baseline brute force algorithm 2,119,301 milliseconds, which is roughly 35.32 minutes. This result is actually from the first time I wrote a brute force algorithm. The brute force result you can find on GitHub, it actually took 1,770,851 milliseconds, which is about 29.51 minutes. I have no idea what I did in the rewrite that shaved off 4 minutes. I won’t even lie to you reader, I am a busy undergraduate student, so I didn’t even do the “best practice” for testing, which would be to quit all other processes and then benchmark it. However, while I was testing, I was doing a Canvas assignment, so I was doing lots of background work.

Normally in science you have to run an experiment at least three times for data accuracy. But you must understand that I don’t want to give up an hour and a half of my life to an algorithm that we’re going to be improving upon anyways.

For fun, I will benchmark the short-circuit and parallel versions as well. Short-Circuited (Basic): 49874ms (50 seconds)

Parallel: 10589ms (10.5 seconds)

For the parallel version, the implementation isn’t perfect. I believe that we don’t prevent false sharing amongst cache lines (ie we get more cache swaps between threads leading to more cache misses), which could explain why on my M1 MacBook Air, we don’t get close to a 8 times speedup. Since this was for fun, I’m not going to perfect the parallelized version, that can an exercise to the reader :)

Chapter 3

DPLL

The Davis-Putnam-Logemann-Loveland (DPLL) algorithm is a complete backtracking-based search algorithm for finding satisfiable assignments for SAT problems. It is, dare I say, the foundational algorithm for modern SAT solvers, as the state-of-the-art algorithm used by basically every SAT solver builds upon the foundations that DPLL provides, as we will see later on.

It consists of three components:

- Recursive backtracking
- Unit propagation
- Pure literal elimination

How does it work?

Imagine a binary tree of variables, each with two branches representing their assignment, one for true and one for false. Suppose we don't know anything about our CNF equation. That is, from looking at it, any variable could have any value, and we don't know how each assignment helps us in finding out whether or not the equation is satisfiable. How can we figure out if we can satisfy the equation without just brute forcing it?

The answer is making an ass out of you and me to make assumptions.

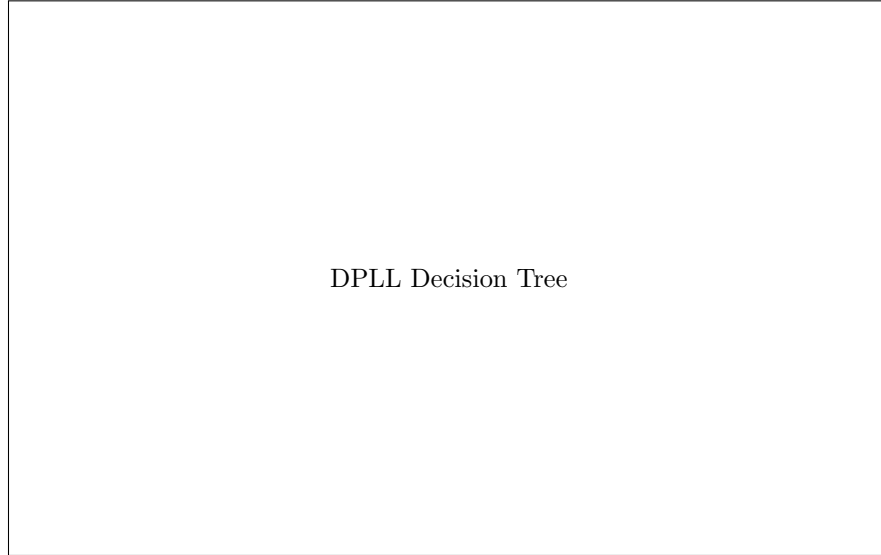


Figure 3.1: Example DPLL decision tree (the outcomes may not correspond to an actual CNF instance, sue me).

Going back to our hypothetical, we don't know anything about anything. We could have a giant decision tree to explore with an exponential number of leaves to evaluate.

How does making assumptions help us?

Do we just run a random number generator and if it returns 1, we go "okay, it's satisfiable," and false otherwise?

No.

We make assumptions about our literal assignments.

Unlike brute forcing where we try every possible combination of variable assignments at once, in DPLL we start with a literal, we'll call it A . There is nothing in the equation that indicates whether or not A should be true or false; either could be valid.

So to tackle our problem, initially we will assume A is true and see where that leads us. We propagate that value throughout our clauses, and suppose there is a clause that can't be satisfied. That's a contradiction, because if the equation was satisfiable then $A = \text{True}$ couldn't have led us there.

So we've figured out that at the very least $A = \text{True}$ leads to a contradiction.

We've now eliminated half of the search space immediately and only need to consider the cases where $A = \text{False}$.

This may not make total sense immediately, so instead of giving one explanation of the algorithm and then building it, we will build up the algorithm and explain how each part works.

¹

¹Note for future me: write a better introduction.

3.1 Recursive Backtracking

First, we need to rewrite our brute force algorithm to be recursive, as that's the foundation for the entire algorithm (duh). To do this, we will introduce a new method *solve()* which will be our recursive solving method. To also make things simpler, we will extract the equation evaluation into its own separate method as well.

This means our SAT solver becomes like so:

```

1  public interface:
2      SATSolver(string input)
3      int getNumVars()
4      int getNumClauses()
5      int[] [] getClauses()
6      optional<bool>[] getSolution()
7      void printSolution()
8      bool solveCNF()
9
10 private interface:
11     void parse(string equation)
12     bool solve(bool[] assignment)
13     bool evaluate(bool[] assignment)
14     int numVars
15     int numClauses
16     int[] [] clauses
17     optional<bool>[] lits

```

The flow essentially becomes like this: *solveCNF()* calls *solve()*, which calls itself over and over and returns whatever solution it finds back up to *solveCNF()*, who will handle any sort of cleanup.

Why can't we just make *solveCNF()* call itself recursively?

Good question.

That's because *solveCNF()* will handle any setup that *solve()* will need in order to function properly, and it will handle any cleanup that *solve()* may create.

Let's now discuss how *solve()* will work.

It will be passed in a *bool[]* for the current assignment of literals. When the size of our assignment is equal to the number of variables, we will then evaluate the satisfiability of the assignment, and if it's satisfiable we return true. Otherwise, we return false, backtrack, retry, until we either find a satisfiable assignment or we've exhausted them all.

The pseudocode is as so, starting with *evaluate()*:

²https://en.wikipedia.org/wiki/DPLL_algorithm

```
1  bool evaluate(bool[] assignment)
2  {
3      bool solved = true;
4
5      for each (clause in clauses)
6      {
7          bool satisfied_clause = false;
8
9          for (j = 1 up to clause.size)
10         {
11             int variable = clause[j];
12             int v_idx = abs(variable);
13
14             if (variable > 0)
15             {
16                 satisfied_clause = assignment[v_idx]
17                     || satisfied_clause;
18             }
19             else
20             {
21                 satisfied_clause = !assignment[v_idx]
22                     || satisfied_clause;
23             }
24
25             if (satisfied_clause == True)
26             {
27                 continue;
28             }
29         }
30
31         solved &= satisfied_clause;
32
33         if (solved == False)
34         {
35             break;
36         }
37     }
38
39     if (solved == True)
40     {
41         lits = assignment;
42     }
43     else
44     {
45         lits = no-value;
46     }
47
48     return solved;
49 }
```

As you can see, all we've done is move our equation evaluation from brute force into its own function. One thing to note is that this method also handles updating our *lits* field. Thus, if *evaluate()* returns true, then *lits* will hold the satisfying assignment, and if it returns false, then *lits* will hold nothing.

Now our recursive *solve()* method:

```
1  bool solve(bool[] assignment)
2  {
3      if (assignment.size == numVars)
4      {
5          return evaluate(assignment);
6      }
7
8      append(assignment, True);
9
10     if (solve(assignment))
11     {
12         return true;
13     }
14
15     assignment.back() = False;
16
17     if (solve(assignment))
18     {
19         return true;
20     }
21
22     truncate(assignment, 1);
23     return false;
24 }
```

This might be slightly confusing at first, so let's go over it.

We first check our base case. If our assignment has all of its variables, we evaluate it and return the result whether it be true or false. And you'll notice that whenever we return true from a call of *solve()*, we immediately return true afterwards, so therefore, if we find a satisfiable assignment, we will go up our call stack back to *solveCNF()*, so no need to worry that it will terminate if we find a satisfiable assignment.

However, if our assignment still needs some variables, we append true to our assignment, then we do a recursive call on *solve()*. If that recursive call leads to a satisfiable assignment, we return true, otherwise, we swap out our last value for false and try *solve()* again. And if that ends up working out, we return true, but if it doesn't then we remove the last value and return false.

So how do we know that it will guarantee termination for an unsatisfiable problem?

Notice how when assigning the current variable true or false and it doesn't work, we just remove the last one and go up the call stack by one? Well suppose we're at the top of the call stack (we've assigned the first variable) and the first variable is assigned false, and that didn't lead to a satisfiable assignment. Then we remove that first variable from the assignment, and then we return false. So our assignment array is empty, and we've returned to *solveCNF()*. Thus, *solve()* will terminate eventually.

And how do we know that we'll try every single assignment?

Well for the very last variable, it's easy to see, if it being true doesn't satisfy the equation, we assign it false, and if that doesn't work, we remove it and go up the call stack. Then for the previous variable, if it was true, we assign it false and go back to the last variable where it tries true and/or false again. Then that doesn't work, we go up the call stack twice, and that continues until we've found a satisfiable assignment or we've exhausted them all.³

Lastly, our *solveCNF()* method:

```
1  bool solveCNF()
2  {
3      bool[1..numVars] assignment;
4
5      return solve(assignment);
6  }
```

Because we're dealing with pseudocode, we don't have to do things such as reserving or resizing the assignment array. And since *evaluate()* takes care of making sure our *lits* field holds a valid assignment (if found) and holds nothing if not, we don't need to do anything else in *solveCNF()*.

You may wonder though, why not pass in *lits* to *solve()* or have no setup and make *solveCNF()* purely recursive?

Those are very valid questions to ask.

This has to do with the differences between pseudocode and actual implementation. In the actual implementation, some methods depend on the state of *lits*, such as whether or not it holds anything, and by passing it into *solve()*, we force it to hold something, even if there is no satisfiable assignment. And it's true, that in *solveCNF()* if we had no setup, we could make it recursive on *lits*, however, I chose to implement it differently.

When you are reading this, please ask yourself these kinds of questions, because oftentimes, the way you read my implementation is not the best way, and even if it is, there is no reason as to why you can't think of different implementations, try them out yourself, and compare. Perhaps, and I expect this will happen often, you will find a better implementation than the one written down here.

³This is a very informal proof, but what's most important is that we build up our understanding of what's going on instead of dumping notation. Perhaps I will do a formal proof in the future.

3.1.1 Recursive Backtracking – Results

Running this code on the same 1000 instances (uf20-91), it took 1,846,644ms, which is roughly 30.78 minutes. This is actually a pretty big improvement from our 35.32 minutes via brute forcing. Although this seems like a big gain in speed, it should be noted that at this point the algorithm is still just a fancier brute forcing algorithm. The next time I rewrote this algorithm, it took 1,540,613ms, which is about 25.68 minutes. So once again, that’s about a 5 minute speedup just from rewriting it. Perhaps we should rewrite every piece of software in the world to make it run 5 minutes faster.

Here’s a question for you, why might assigning our variables true instead of false first lead to an increase in speed?

Hint: Look at the number of negations present in the testing set.

3.2 Unit Propagation

Unit propagation is the next step in implementing DPLL, and it introduces the idea of implication.

Imagine clause: $A \rightarrow Z$, and we know that $A = \text{False}$. What can we say about Z ?

Well, if our equation is satisfiable, then the clause must be satisfiable, therefore $Z = \text{True}$ because if it didn’t, that would be a contradiction.

So instead of assigning variables $B, C, \dots, X$, then assigning Z , why don’t we just do it now, and use that knowledge in other clauses containing Z or $\neg Z$?

That is essentially the idea behind unit propagation: the decisions we make imply other assignments, and so we should force them now instead of waiting until later.

However, this introduces a light hiccup into our implementation.

You see, our initial invariant about assigning variables is that they are assigned in numerical order instead of (possibly) out of order.

“Not a problem! We’ll just make the assignment array have space for every variable so we can index it anywhere,” I hear you think.

Unfortunately that interferes with our base case because it relies on the size of the assignment array. If it is initially at a fixed size of *numVars*, then our base case will always fire because our assignment array always has a size of *numVars*. So we would either need to change our base case or change how we store learned variables.

What about a learned set? One that holds whether or not we’ve learned a variable (positive or negative), and then as we traverse down the call stack, we just check if a variable is already learned, and if so, we give it its learned value?

That would 100% work.

In fact, the first time I implemented DPLL, that is exactly what I used.

However, it has the disadvantage of being annoying to maintain.

Instead, why don’t we create a new data type called something like *var* that represents the state of each variable: *True*, *False*, and *Unassigned*.

Then we can just maintain a simple counter of how many are not *Unassigned*, and then that keeps our base case nice and neat.

Yes dear reader, we are going to do that, however, if you'd like to use the learned set, reading about how we do it without a learned set will help you and offer clues on how to implement it your way.

3.2.1 Changes To Our Implementation

```

1      public interface:
2          SATSolver(string input)
3          int getNumVars()
4          int getNumClauses()
5          int[] [] getClauses()
6          optional<bool>[] getSolution()
7          optional<var>[] getSolution()
8          void printSolution()
9          bool solveCNF()
10
11     private interface:
12         enum var
13         {
14             False
15             True
16             Unassigned
17         }
18
19         void parse(string equation)
20         bool solve(bool[] assignment)
21         bool solve(var[] assignment, int level)
22         bool evaluate(bool[] assignment)
23         bool evaluate(var[] assignment)
24         void propagate()
25
26         int numVars
27         int numClauses
28         int numAssigned
29
30         int[] [] clauses
31
32         optional<bool>[] lits
33         optional<var>[] vars

```

Here we just add in a new enum for the states that a variable can be, and adjust the methods using *bools* to use the new enum type instead.

3.2.2 “Textbook” Unit Propagation

Before we start changing our algorithm, let's implement the *propagate()* method.

As stated before, the idea is that given a clause with either only one variable or only one *Unassigned* variable (and the rest evaluate to *False*), we can immediately say that *Unassigned* variable will be whatever value it needs to be to evaluate to *True* for that clause.

In the “textbook” way of implementation,⁴ once a clause is satisfied because of a literal, you remove every other clause that is satisfied by that literal. And for every other clause containing the negation of the literal, you remove that negation entirely. This is done to shrink the search space.

We will implement this algorithm with as much malicious compliance as you can carry at an office job without getting fired:

```

1      bool solveCNF() {
2          var[1..numVars] assignment
3
4          for each (var in assignment) {
5              var = Unassigned
6          }
7
8          propagate(assignment)
9          solve(assignment, 0)
10     }
```

We want to propagate before we start solving in case there are clauses with only one variable (it’s already unit).

```

1      bool solve(bool[1..numVars] Assignment, int level) {
2          if (level == numVars - 1) {
3              return evaluate(Assignment)
4          }
5
6          bool[1..N] backupAssignment = assignment
7          int[][] backupClauses = clauses
8
9          if (Assignment[level] == Unassigned) {
10             Assignment[level] = True
11             propagate(Assignment)
12         }
13
14         if (solve(Assignment, level + 1)) {
15             return True
16         }
17
18         clauses = backupClauses
19         Assignment = backupAssignment
20
21         if (Assignment[level] == Unassigned) {
22             Assignment[level] = False
```

⁴https://en.wikipedia.org/wiki/Unit_propagation

```

23         propagate(Assignment)
24     }
25
26     if (solve(Assignment, level + 1)) {
27         return True
28     }
29
30     return False
31 }

```

Before we get onto the code for unit propagation, let's understand this code first. Because unit propagation no longer guarantees that variables are assigned in order, we need to pass in a “level” as a parameter so we know which variable we need to assign. We also now need to check that propagation didn't already assign this variable. If it's currently Unassigned, we need to assign it then propagate, but if it's been assigned before we assign it, then we know that it's the result of propagation, so we can continue onto the next level. Other than that, it's practically the same as our recursive backtracking code, except now we need to have a “backup” of our clauses and our assignment. Why? Because *propagate()* **mutates** the clauses and the assignment in a way that breaks the assumption that our literals are assigned in “numeric” order. Since it returns nothing, we don't have a way to undo that easily, so instead, for readability, we just have a backup copy and whenever propagation leads to an unsatisfying assignment, we return false and undo whatever changes we made by copying back into our solver's fields. And the reason why we don't need to undo the propagation before returning False is because when we move back up the call stack by one level, the method continues by undoing the effects for us. Now let's look at how we perform textbook unit propagation:

```

1     void propagate(int[1..N] Assignment) {
2         while (true) {
3             Set<int> unitClauses;
4             Set<int> unitVars;
5
6             for (i = 0 up to numClauses) {
7                 clause = clauses[i]
8
9                 bool isUnit = True
10                int unassignedIdx = -1
11                int numUnassigned = 0
12
13                for (j = 0 up to clause.size) {
14                    int var = clause[j]
15                    int idx = abs(var)
16
17                    if (Assignment[idx] == Unassigned) {
18                        numUnassigned = numUnassigned + 1
19                        unassignedIdx = j

```

```

20         } else if (var > 0 && clause[j] == True || var <
    0 && clause[j] == False) {
21             isUnit = false
22         }
23     }
24
25     isUnit = isUnit && numUnassigned == 1
26
27     if (isUnit) {
28         add(unitClauses, i)
29         add(unitVars, clause[unassignedIdx])
30
31         int var = clause[unassignedIdx]
32         int idx = abs(var)
33
34         if (var > 0) {
35             assignment[idx] = True
36         } else {
37             assignment[idx] = False
38         }
39     }
40 }
41
42 int clauseIdx = 0;
43 for each (clause in clauses) {
44     for each (var in clause) {
45         if (contains(unitVar, -var)) {
46             remove(clause, -var)
47         }
48
49         if (contains(unitVar, var) && contains(unitClause
    , clauseIdx)) {
50             remove(candidates, clause)
51         }
52     }
53
54     clauseIdx = clauseIdx + 1
55 }
56
57 if (unitClauses.size == 0 && unitVars.size == 0) {
58     break
59 }
60 }
61 }

```

Okay, now let's understand what's happening here. We immediately go into an infinite loop (nice), and we start looking at each clause for two things:

1. The clause has only one Unassigned variable

2. Every other literal in the clause is False (remember, literals are evaluated variables)

If both things are true, then the Unassigned variable will be assigned whatever value is required to make it evaluate to True within that clause, and we know we have found a unit variable and a unit clause. After we have found all of our unit variables and unit clauses, we then rescan every other clause and delete the each unit variable's negative, (if we learn that -5 is unit, then we remove +5), and if the clause contains a unit variable, we just delete that clause entirely from the database. Once we stop finding new unit variables and new unit clauses, we stop entirely. You may wonder (I reread my code and thought I found a bug), "how do we know that we always terminate?" That's a great question. You see, this confusion may come from the fact that we rescan the database every iteration of the *while()* loop searching for unit clauses. What is stopping us from re-finding the same clause as unit upon a new iteration? The answer is much simpler than you'd expect: since we assign our unit variables, the clauses upon the next iteration no longer follow our criteria for being unit propagate-able because there is now a literal in that clause that is True. Therefore, the number of unit variables and clauses monotonically decrease between iterations (a fancy way of saying the number can only go down).

Now, you may be wondering, "what's the performance of our solver now?" Let me answer that as plainly as possible: worse than brute force. How? Easy! We are doing SO MANY things that hurt the computer. We are allocating TONS of memory in order to backtrack, we are allocating even more memory to propagate (hash sets aren't cheap), and hashing, while $O(1)$, is not a cheap operation to do. So yeah, I can believe that it's worse than brute force because at least the brute force algorithm was easy on the CPU. Let's learn how this is *usually* ⁵ implemented!

3.3 Better Unit Propagation

So the textbook style unit propagation sucked. Let's change that. As discussed before, the biggest bottleneck/inefficiency in the actual implementation is the number of copies we need to make of our clauses and assignments on each decision level. Assigning one variable creates a new decision level, so 20 variables means 20 levels. For an unsatisfiable CNF equation, I estimate we need to make about 3 copies of our clauses and assignments: one for the initial storage, another for the first failure, and a third for the second. Even if we ignore the inefficiencies in how we propagate (memory allocations, traversals, and adjustments our arrays would need to make when we delete things), this is a huge amount of memory to store. If you think that 20 variables and 91 clauses is a lot, modern SAT solvers can handle equations with **millions** of variables,

⁵I wanted to say "actually implemented", however, that's a big generalization, and again, since I'm not the expert here, I don't think I have the authority to say that. The next section is how *I* implemented it, and it roughly follows the idea on how SAT solvers implement this.

so creating an algorithm that makes brute force look attractive is actually an achievement.⁶

So what's the strategy? Firstly, it would be nice if we didn't actually mutate our clauses at all. Ie, no clause deletion, and no variable deletion. Next, it would be nice if we didn't need hash sets in order to store our unit variables. Instead, it would be nice if we had a data structure where we put in our decided variables, and then it slowly shrinks as we iterate through. Ie, we propagate one on variable at a time and never again. This sounds like we need a *queue*.⁷ Lastly, instead of copying our assignment, and constantly reiterating through the *entire* clause database, what if we just marked which variables we assigned, and which clauses we've satisfied? That way when we need to undo, we just mark those variables as Unassigned, and those clauses as no longer satisfied. That way we save a lot of memory (and besides, copying is NOT $O(1)$ as you might be led to believe by that deceptively simply "=" operator).

Let's change our interface:

```

1      public interface:
2          SATSolver(string input)
3          int getNumVars()
4          int getNumClauses()
5          int[] [] getClauses()
6          optional<bool>[] getSolution()
7          optional<var>[] getSolution()
8          void printSolution()
9          bool solveCNF()
10
11     private interface:
12         enum var
13         {
14             False
15             True
16             Unassigned
17         }
18
19         void parse(string equation)
20         bool solve(var[] assignment, int level)
21         bool evaluate(var[] assignment)
22         (int[] vars, int[] clauses) propagate()
23
24         int numVars
25         int numClauses
26         int numAssigned
27
28         int[] [] clauses
29         bool[] satisfied
30

```

⁶They might revoke my degree for that actually

⁷That wasn't shoehorned in at all!


```

31         optional<bool>[] lits
32         optional<var>[] vars

```

We need to make some minimal changes to *solveCNF()*:

```

1     bool solveCNF() {
2         var[1..numVars] assignment
3
4         for each (var in assignment) {
5             var = Unassigned
6         }
7
8         propagate(assignment, 0)
9         solve(assignment, 0)
10    }

```

Since *propagate()* now takes a decision level/variable index, we need to pass one to the method. Although no variable has an index of 0 (because it's reserved to mark the end of a clause), passing in 0 just tells it “Hey, there isn't a literal to propagate on, so just find all clauses with only one variables and mark them as satisfied and ‘learn’ those variables.” And since this is at the root level, we don't need to undo anything if it turns out that the equation is unsatisfiable, since we're gathering things that HAVE to be a certain value if this equation is to be satisfiable.

Since we don't need to change *solveCNF()*, we can go straight to *solve()*:

```

1     bool solve(int[1..N] Assignment, int level) {
2         if (level == numVars - 1) {
3             return evaluate(Assignment);
4         }
5
6         if (Assignment[level] != Unassigned) {
7             return solve(Assignment, level + 1)
8         }
9
10        Assignment[level] = True
11        (int[] unitVars, int[] satisfiedClauses) = propagate(
12            Assignment, level)
13
14        if (solve(Assignment, level + 1) == True) {
15            return True
16        }
17
18        for each (var in unitVars) {
19            int idx = abs(var)
20            Assignment[idx] = Unassigned
21        }

```

```

22         for each (clauseIdx in satisfiedClauses) {
23             satisfied[clauseIdx] = False
24         }
25
26         Assignment[level] = False
27         (int new_unitVars, int new_satisfiedClauses) = propagate(
Assignment, level)
28
29         if (solve(Assignment, level + 1) == True) {
30             return True
31         }
32
33         for each (var in new_unitVars) {
34             int idx = abs(var)
35             Assignment[idx] = Unassigned
36         }
37
38         for each (clauseIdx in new_satisfiedClauses) {
39             satisfied[clauseIdx] = False
40         }
41
42         Assignment[level] = Unassigned
43         return False
44     }

```

This method is practically the same, except now, we just check if we've already assigned the variable at the current decision level. If we have assigned a variable before we reach its decision level, then it must have been inferred/learned from the *propagate()* method, so we don't need to call it again. Then, it's business as usual: assume the current variable is True, propagate, if it leads to a satisfiable assignment, return True, otherwise, we undo what we've assigned and marked as learned, then repeat with False.

What's more interesting is our shiny new *propagate()* method:

```

1     (int[] vars, int[] clauses) propagate(int[] assignment, int level
) {
2         queue<int> unit
3
4         int[] unitVars
5         int[] satisfiedClauses
6
7         unit.enqueue(level)
8
9         while (unit.size > 0) {
10             unit.dequeue()
11
12             for (i = 0 up to numClauses) {
13                 if (satisfied[i]) {
14                     continue

```

```

15         }
16
17         clause = clauses[i]
18
19         bool isUnit = True
20         int unassignedIdx = -1
21         int numUnassigned = 0
22
23         for (j = 0 up to clause.size) {
24             int var = clause[j]
25             int idx = abs(var)
26
27             if (Assignment[idx] == Unassigned) {
28                 numUnassigned = numUnassigned + 1
29                 unassignedIdx = j
30             } else if (var > 0 && clause[j] == True || var <
0 && clause[j] == False) {
31                 isUnit = false
32
33                 satisfied[i] = true
34                 append(satisfiedClauses, i)
35             }
36         }
37
38         isUnit = isUnit && numUnassigned == 1
39
40         if (isUnit) {
41             add(unitClauses, i)
42             add(unitVars, clause[unassignedIdx])
43
44             int var = clause[unassignedIdx]
45             int idx = abs(var)
46
47             if (var > 0) {
48                 assignment[idx] = True
49             } else {
50                 assignment[idx] = False
51             }
52
53             unit.enqueue(var)
54
55             append(unitVars, var)
56             satisfied[i] = true
57             append(satisfiedClauses, i)
58         }
59     }
60 }
61
62 return (unitVars, satisfiedClauses)
63 }

```

So, let's discuss what's going on. *propagate()* now takes in the decision level, which is also the same thing as an index for a variable (conceptually speaking). It then adds that to a queue, then while the queue is not empty, it scans the clauses as before. First it makes sure we're not operating on a clause that's already been satisfied (because we can't tell anything from it), and when it finds a clause that has a true literal, it marks the clause as satisfied. Then when it finds a unit clause, it marks the clause as satisfied, keeps track of the new unit variable it's found, and then adds that unit variable to the queue. Then once it's done, it returns the unit variables and the indices of the satisfied clauses.

You may have some questions about this approach. Firstly, how do we make sure we don't ever add the same variable twice? The answer is the same as before: because we assign a variable once it's found to be in a unit clause, it can't be found as Unassigned again, and therefore can't be marked as a unit variable again. Then, since we skip over already satisfied clauses, the amount of clauses we scan each iteration decreases monotonically.

And going back to our *solve()* code, even if we mark the same clause as satisfied or the same variable as unit twice, it doesn't matter because when we backtrack, we simply restore it back to its state of unsatisfied or Unassigned, respectively. What's important is that we don't mark the same clause or variable twice *across decision levels* because then, and only then, would our backtracking code become a problem because we wouldn't be correctly restoring the state of the equation back to what it was before the decision level was entered. But since we can't mark the same variable or clause twice **until** we backtrack, there's nothing to worry about.

Before we end this section, you may also be wondering: can we use the *satisfied* array in our *evaluate()* method to skip clauses we know we've already satisfied? And the answer is yes, and I would encourage you to do so in your implementations.

3.3.1 Better Unit Propagation - Results

So after implementing better unit propagation, this took 868ms to solve all 1000 satisfiable equations. The first time I implemented this version of unit propagation, it actually took 2363ms, so this is somehow over twice as fast. And because it's so fast, just like the first time I implemented this, we will be now testing our solver on unsatisfiable equations as well.

Our second testing set comes from the same database as before.⁸ And our next testing set is called "UUF50.218.1000." In other words, it's full of equations that have 50 variables, 218 clauses, and there are 1000 such equations in the set. In order to solve all 1000 equations, it took 140,411ms (2.34 minutes). That's also much faster than the first time I implemented this algorithm. The first time I did, it took 646,228ms (10.77 minutes). Honestly, with this much

⁸<https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html>, in case you needed a reminder.

speedup, I don't know if I was just bad at implementing these algorithms the first time,⁹ or if downgrading my operating system (and subsequently wiping my harddrive clean)¹⁰ just has that much of an effect on execution speed.¹¹

Why didn't we test our previous versions out on unsatisfiable equations? That's a good question. For brute forcers, since there is no satisfiable assignment, it needs to check every single one to conclude that it's unsatisfiable, which would take a lot of time. Likewise with our recursive backtracking solver. And since our (my) implementation of textbook unit propagation was *worse* than brute force,¹² obviously we wouldn't want to test it on something that will take more time than a satisfiable equation. Also because the asymptotic run-time is *still* exponential, having a much larger equation would take a much, much larger amount of time, even if they were satisfiable.

3.4 Pure Literal Elimination

3.5 Optimizing Our Current Approach

3.6 Iterative DPLL

⁹Highly likely

¹⁰I HATED MacOS Tahoe. Sequoia is where it's at.

¹¹Also likely, depending on how much RAM and swap is being used to begin with.

¹²A difficult feat.